

DevLog — трекер задач для інженерних команд

Про завдання

Потрібно зібрати **DevLog** — простий веб-додаток для відстеження задач в інженерній команді. Сам продукт нескладний, але в ньому має бути вбудований **AI-агент**, який допомагає автоматично розбивати задачі, розставляти пріоритети та формувати статус-апдейти.

Головне, на що ми дивимось:

і Наскільки природно ти використовуєш coding agent у власному процесі розробки — і наскільки продумано AI-функції вбудовані в сам продукт.

Ми не очікуємо готовий SaaS. Нам важливі чіткі рішення, чесні компроміси і відчуття, що ти використовуєш AI-інструменти як підсилювач, а не як заміник мислення.

Контекст

Команда розробників використовує DevLog для відстеження поточних задач. Кожна задача має назву, опис, статус і пріоритет. Головний біль команди — вручну розбиратись, що робити далі, писати підзадачі та складати апдейти для колег. Це займає час і дратує.

Твоє завдання — зробити базовий трекер і додати поверх нього AI-шар, який прибирає цей friction.

Що треба реалізувати

Базовий трекер

1 CRUD для задач

Створення, перегляд, редагування і видалення задач. Обов'язкові поля: title, description, status (todo / in-progress / done), priority (low / medium / high), createdAt.

2 Список задач

Відображення всіх задач з фільтрацією за статусом і сортуванням за пріоритетом або датою. Без зайвих прикрас — головне, щоб працювало зручно.

3 Зберігання даних

Задачі мають зберігатись між перезавантаженнями сторінки. Підійде SQLite, JSON-файл або in-memory store — але обов'язково поясни свій вибір і його обмеження у README.

AI-функції — реалізуй щонайменше 2

Важливо:



Це має бути справжній агент, а не просто один запит до LLM. Агент повинен самостійно приймати рішення або виконувати кілька кроків на основі контексту задач.

A

Агент пріоритизації

Агент аналізує поточний список задач і пропонує, з чого почати день, — з поясненням логіки. Він має враховувати пріоритет, вік задачі, статус, а не просто повертати першу за рангом.

B

Декомпозиція задачі

За назвою і описом задачі агент генерує структурований список підзадач і за бажанням одразу створює їх у додатку. Якщо задача сформульована нечітко — агент має уточнити перед генерацією.

C

Генератор статус-апдейту

Агент складає короткий асинхронний апдейт у стилі повідомлення в командний Slack — на основі задачі, її підзадач або нотаток про виконання. Тон має відповідати контексту.

D

Власна ідея

Запропонуй і реалізуй свою агентну функцію. Додай абзац з поясненням, чому це корисно для інженерної команди. Тут є простір для того, щоб здивувати.

Технічний стек

- Фронтенд: **Next.js** — вибір роутингу, стейт-менеджменту і стилізації за тобою.
- Бекенд: **Node.js з Express** або Next.js API routes. REST чи tRPC — як зручно.
- AI: будь-який LLM API (Anthropic, OpenAI тощо). Якщо не хочеш ділитись ключами — можна замокати виклики, але обов'язково задокументуй це.
- Авторизація не потрібна. Один користувач, одна команда.
- Деплой не потрібен. Достатньо, щоб все запускалось локально через `npm install && npm run dev`.

Про ключі:



Додай `.env.example` з усіма необхідними змінними, але без реальних значень. Ми підставимо свої ключі при перевірці AI-функцій.

Coding agent — це частина оцінки

Ми хочемо бачити, як ти працюєш разом з AI coding agent (Claude, Cursor, Copilot тощо) — це явна частина завдання, а не просто бонус.



UI — через агента

Очікуємо, що фронтенд scaffolded агентом: компоненти, лейаут, стилізація. Не обов'язково все до останнього пікселя — ручні правки нормальні і навіть очікувані. В `AGENT_LOG.md` поясни, що саме генерував агент, що довелося змінити і чому.

Задokumentуй свій процес



Додай файл `AGENT_LOG.md` у корінь репо. Напиши: які агенти використовував, які задачі їм давав, де вони допомогли, а де довелося втручатися або переробляти. Чесність тут важливіша за красиву подачу.

Що треба здати

- **GitHub репо** — публічне або з доступом для нашої команди. У `README.md` — інструкція із запуску і коротко про архітектуру.
- **`AGENT_LOG.md`** — чесна розповідь про те, як ти працював з агентом.
- **`.env.example`** — всі необхідні змінні середовища без реальних значень.
- **Робочий локальний запуск** — `npm install && npm run dev` має стартувати без помилок.
- **Опціонально: Loom** — 3–5 хвилин огляду продукту і AI-функцій. Буде плюсом, але не обов'язково.

Як оцінюємо

Критерій	На що дивимось
Робота з AI-агентом	Чи scaffolded агентом UI? Чи є in-product функції справді агентними — кілька кроків, а не один промпт?
Якість коду	Читабельність, структура, нормування назв, розподіл відповідальностей. Без очевидних антипатернів.
Продуктове мислення	Чи AI-функції справді корисні, а не для галочки? Чи є конкретні рішення за якими стоїть думка?
Комунікація компромісів	Якість <code>README</code> і <code>AGENT_LOG</code> . Чи можеш пояснити, що свідомо не робив і чому?
Full-stack реалізація	Чи коректно працює API і зберігання даних? Чи фронтенд нормально з ним взаємодіє?
UI / UX	Чи зрозумілий UX, гарний UI

Про скоп:



Якщо витрачаєш більше 8–10 годин — щось пішло не так. Краще прибрати одну фічу і пояснити чому, ніж розмазати якість по всьому.

Здача

Посилання на репо надішли своєму контакту з нашого боку. Якщо в процесі виникнуть питання — пиши, не соромся. Ми дивимось і на те, як ти комунікуєш під час роботи.

Удачі. Чекаємо з цікавістю.